Guide Complet de Realisation — v2.0

Systeme Multi-Agents IA Vocal

avec RAG Agentique et Orchestration

Node.js + FastAPI + LangChain + RAGFlow + MySQL + Orchestration

Projet Academique | 2024

Nouveautes de la version 2

> 💡 *Cette version ajoute l'orchestration intelligente entre RAGFlow (documents) et MySQL (Excel) — le coeur du RAG agentique.*

Composant	Version 1	Version 2
Tools	2 tools separés	3 tools + orchestration
Décision source	Agent decide seul	Router explicite + mots-cles
Combinaison sources	Non presente	Tool recherche_combinee
System Prompt	Simple	Algorithme de decision
Gestion echec	Basique	Fallback intelligent
Schema SQL	Non expose	Expose au LLM

1. Architecture et flux complet

1.1 Vue globale

Couche	Technologie	Role	Statut
Interface vocale	Node.js + React	STT, TTS, UI	Existant
API Bridge	FastAPI Python	Pont Node.js/Python	A creer
Orchestrateur	LangChain + Groq	Router + Agent RAG	A creer
Documents	RAGFlow + vecteurs	PDF et Word vectorises	A creer
Donnees SQL	MySQL + pymysql	Excel converti en tables	A creer
LLM	Groq llama-3.3-70b	Raisonnement + synthese	Gratuit

1.2 Flux avec orchestration

1. Utilisateur parle → STT Node.js → texte
2. Node.js → POST /ask → FastAPI Python
3. FastAPI → Agent LangChain
4. Agent ANALYSE la question
5. Router decide : Documents / SQL / Les deux / LLM seul
6. Execution du ou des tools choisis
7. Agent SYNTHETISE les resultats
8. Reponse → FastAPI → Node.js → TTS → voix

1.3 Logique de decision du Router

Type de question	Source choisie	Exemple
Qualitative / textuelle	RAGFlow (documents)	Que dit le rapport sur X ?
Quantitative / chiffres	MySQL (Excel)	Combien de clients en 2024 ?
Mixte texte + chiffres	RAGFlow + MySQL	Le produit du rapport, son CA ?
Question generale	LLM seul	Qu'est-ce que Python ?
Aucune source trouvee	Fallback LLM	Repond avec ses connaissances

2. Preparation de l'environnement

2.1 Installer les packages Python

```
pip install fastapi uvicorn langchain langchain-groq
pip install langchain-community sentence-transformers
pip install faiss-cpu pymysql sqlalchemy pandas openpyxl
pip install python-dotenv langchain-text-splitters
pip install python-multipart requests
```

2.2 Structure du projet

```
mon_projet/
├── node_app/              <- app Node.js existante
│   ├── server.js
│   ├── agent.js           <- a ajouter
│   └── public/app.js
│
├── python_backend/        <- a creer
│   ├── main.py            <- API FastAPI
│   ├── tools.py           <- 3 tools + orchestration
│   ├── agents_config.py   <- config multi-agents
│   ├── excel_to_mysql.py  <- conversion Excel
│   ├── .env               <- cles API
│   └── documents/         <- PDF et Word
│
└── ragflow/               <- RAGFlow Docker
```

3. Installation RAGFlow

3.1 Lancer RAGFlow avec Docker

> 💡 *RAGFlow gere toute la vectorisation. Son API remplace FAISS ou ChromaDB dans ce projet.*

```
git clone https://github.com/infiniflow/ragflow.git
cd ragflow
docker compose up -d
# Interface : http://localhost
```

3.2 Configuration RAGFlow

9. Ouvrir http://localhost dans le navigateur
10. Creer un compte administrateur
11. Model Providers → Ajouter Groq avec ta cle API
12. Datasets → Creer un nouveau dataset
13. Uploader tes fichiers PDF et Word
14. Lancer le parsing et la vectorisation automatique
15. API Keys → Generer une cle API RAGFlow
16. Copier le Dataset ID depuis l'interface

3.3 Tester RAGFlow

```
import requests

r = requests.post(
    'http://localhost/v1/retrieval',
    headers={'Authorization': 'Bearer TON_API_KEY'},
    json={
        'question': 'test de recherche',
        'dataset_ids': ['TON_DATASET_ID'],
        'top_k': 3
    }
)
print(r.json())  # doit retourner des chunks
```

4. Conversion Excel vers MySQL

4.1 Script de conversion automatique

```
# python_backend/excel_to_mysql.py
import pandas as pd
import sqlalchemy
import glob, os

engine = sqlalchemy.create_engine(
    'mysql+pymysql://root:@localhost:3306/ta_base'
)

for file_path in glob.glob('./documents/*.xlsx'):
    table_name = os.path.basename(file_path)
    table_name = table_name.replace('.xlsx','').replace(' ','_').lower()
    df = pd.read_excel(file_path)
    df.columns = [c.replace(' ','_').lower() for c in df.columns]
    df.to_sql(table_name, engine, if_exists='replace', index=False)
    print(f'Table {table_name} : {len(df)} lignes importees')

print('Import Excel termine !')
```

4.2 Executer la conversion

```
python excel_to_mysql.py
# Verifier dans PHPMyAdmin : http://localhost/phpmyadmin
```

5. Orchestration — Le coeur du projet

C'est l'etape la plus importante. Elle definit comment l'agent choisit intelligemment entre les documents vectorises et la base SQL.

5.1 Les 3 tools avec descriptions precises

```
# python_backend/tools.py
from langchain_core.tools import tool
from langchain_community.utilities import SQLDatabase
from langgraph.runtime import get_runtime
from dataclasses import dataclass
import requests, os

# ==========================================
# CONTEXTE RUNTIME
# ==========================================
@dataclass
class RuntimeContext:
    db: SQLDatabase

# ==========================================
# TOOL 1 — Documents vectorises (RAGFlow)
# ==========================================
@tool
def recherche_documents(question: str) -> str:
    """Recherche dans les documents PDF et Word vectorises via RAGFlow.
    Utilise cet outil pour :
    - Contenu textuel : rapports, procedures, methodologies
    - Informations qualitatives et narratives
    - Descriptions, explications, definitions
    EXEMPLES : 'Que dit le rapport sur X ?'
    NE PAS utiliser pour des chiffres ou statistiques."""
    try:
        response = requests.post(
            'http://localhost/v1/retrieval',
            headers={
                'Authorization': f'Bearer {os.getenv("RAGFLOW_API_KEY")}'
            },
            json={
                'question': question,
                'dataset_ids': [os.getenv('RAGFLOW_DATASET_ID')],
                'top_k': 3
            }
        )
        data = response.json()
        chunks = data.get('data', {}).get('chunks', [])
        if not chunks:
            return 'AUCUN_RESULTAT: Aucun document pertinent trouve.'
        return '\n---\n'.join([c['content'] for c in chunks])
```

```
    except Exception as e:
        return f'ERREUR_RAGFLOW: {e}'
```

```
# ==========================================
# TOOL 2 — Base MySQL (fichiers Excel)
# ==========================================
@tool
def recherche_sql(question: str) -> str:
    """Recherche dans la base de donnees SQL issue des fichiers Excel.
    Utilise cet outil pour :
    - Donnees chiffrees : montants, totaux, statistiques
    - Listes structurees provenant de tableaux Excel
    - Comparaisons quantitatives et agregations
    EXEMPLES : 'Quel est le CA de X ?', 'Combien de clients ?'
    IMPORTANT : Genere UNIQUEMENT des requetes SELECT.
    La question doit etre une requete SQL valide."""
    runtime = get_runtime(RuntimeContext)
    db = runtime.context.db
    try:
        schema = db.get_table_info()
        result = db.run(question)
        return f'SCHEMA_TABLES:\n{schema}\n\nRESULTAT:\n{result}'
    except Exception as e:
        return f'ERREUR_SQL: {e} | Verifie la requete SQL.'
```

```
# ==========================================
# TOOL 3 — Combinaison des deux sources
# ==========================================
@tool
def recherche_combinee(question: str) -> str:
    """Recherche simultanement dans les documents ET la base SQL.
    Utilise cet outil quand la question necessite
    a la fois du contenu textuel ET des donnees chiffrees.
    EXEMPLES :
    - 'Le produit mentionne dans le rapport, quel est son CA ?'
    - 'Selon le document, combien de clients sont concernes ?'
    - 'Le projet decrit, quels sont ses chiffres ?'
    Cet outil combine automatiquement les deux resultats."""
    result_docs = recherche_documents.invoke(question)
    result_sql = recherche_sql.invoke(question)
    return f'''
SOURCE DOCUMENTS (RAGFlow) :
{result_docs}

SOURCE DONNEES SQL (Excel/MySQL) :
{result_sql}
    '''
```

5.2 System Prompt d'orchestration

C'est le cerveau de l'orchestration. Il guide precisement l'agent dans ses choix.

```
SYSTEM_PROMPT = '''
Tu es un assistant vocal intelligent avec acces a 3 sources.

=== TES SOURCES ===

1. recherche_documents
   Pour : rapports, procedures, descriptions textuelles
   Mots-cles : rapport, document, procedure, methode,
               description, expliquer, definir, selon

2. recherche_sql
   Pour : chiffres, stats, listes issues d'Excel
   Mots-cles : combien, total, CA, montant, liste,
               nombre, statistique, chiffre, moyenne

3. recherche_combinee
   Pour : questions qui melangent texte ET chiffres
   Mots-cles : Les deux types dans la meme question

=== ALGORITHME DE DECISION ===

1. Analyse la question
2. Identifie les mots-cles
3. Choix :
   - Mots textuels seulement → recherche_documents
   - Mots chiffres seulement  → recherche_sql
   - Les deux types           → recherche_combinee
   - Question generale        → reponds directement

=== GESTION DES ECHECS ===

- Si AUCUN_RESULTAT → essaie l'autre source
- Si ERREUR_SQL → reformule la requete
- Si aucune source repond → reponds avec ta connaissance

=== REGLES VOCALES ===

- Maximum 3 phrases dans ta reponse
- Pas de markdown, listes ou symboles
- Parle naturellement comme a l'oral
- Cite la source utilisee brievement
'''
```

6. Backend FastAPI avec orchestration

6.1 Fichier main.py complet

```
# python_backend/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from langchain.agents import create_agent
from langchain_groq import ChatGroq
from langchain_community.utilities import SQLDatabase
from langgraph.checkpoint.memory import MemorySaver
from dotenv import load_dotenv
import os

from tools import (
    recherche_documents,
    recherche_sql,
    recherche_combinee,
    RuntimeContext,
    SYSTEM_PROMPT
)

load_dotenv()
app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_methods=['*'],
    allow_headers=['*'],
)

# Connexion MySQL
db = SQLDatabase.from_uri(os.getenv('DATABASE_URL'))

# LLM Groq
llm = ChatGroq(
    model='llama-3.3-70b-versatile',
    api_key=os.getenv('GROQ_API_KEY')
)

# Memoire par session
memory = MemorySaver()

# Agent avec les 3 tools
agent = create_agent(
    model=llm,
    tools=[
        recherche_documents,
```

```
        recherche_sql,
        recherche_combinee,
    ],
    system_prompt=SYSTEM_PROMPT,
    context_schema=RuntimeContext,
    checkpointer=memory,
)

class QuestionRequest(BaseModel):
    question: str
    agent_id: str = 'default'
    session_id: str = 'session_1'

@app.post('/ask')
async def ask(request: QuestionRequest):
    try:
        config = {
            'configurable': {
                'thread_id': f'{request.agent_id}_{request.session_id}'
            }
        }
        result = agent.invoke(
            {'messages': request.question},
            context=RuntimeContext(db=db),
            config=config
        )
        answer = result['messages'][-1].content
        return {
            'answer': answer,
            'agent_id': request.agent_id,
            'session_id': request.session_id
        }
    except Exception as e:
        return {
            'answer': 'Je rencontre une difficulte,
                       pouvez-vous reformuler ?',
            'error': str(e)
        }

@app.get('/health')
def health():
    return {'status': 'ok', 'sources': ['ragflow', 'mysql', 'llm']}
```

6.2 Fichier .env

```
GROQ_API_KEY=gsk_ta_cle_groq
RAGFLOW_API_KEY=ragflow_ta_cle
RAGFLOW_DATASET_ID=ton_dataset_id
DATABASE_URL=mysql+pymysql://root:@localhost:3306/ta_base
```

6.3 Lancer le backend

```
cd python_backend
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

7. Integration Node.js

7.1 Module agent.js

```
// node_app/agent.js
const fetch = require('node-fetch');
const BACKEND = 'http://localhost:8000';

async function askAgent(question, agentId, sessionId) {
    try {
        const response = await fetch(`${BACKEND}/ask`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                question,
                agent_id: agentId || 'default',
                session_id: sessionId || 'session_1'
            })
        });
        const data = await response.json();
        return data.answer;
    } catch (error) {
        console.error('Erreur agent:', error);
        return 'Desole, je rencontre un probleme technique.';
    }
}
module.exports = { askAgent };
```

7.2 Flux STT → Orchestration → TTS dans le navigateur

```
// node_app/public/app.js
const recognition = new webkitSpeechRecognition();
recognition.lang = 'fr-FR';
recognition.continuous = false;

// Quand l'utilisateur parle
recognition.onresult = async (event) => {
    const question = event.results[0][0].transcript;
    console.log('STT:', question);

    // Afficher 'chargement...'
    document.getElementById('status').textContent = 'Reflexion...';

    // Envoyer a l'agent
    const response = await fetch('/ask-agent', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            question: question,
            agentId: currentAgentId,
```

```
            sessionId: currentSessionId
        })
    });

    const data = await response.json();
    console.log('Reponse:', data.answer);

    // TTS — lire la reponse
    const utterance = new SpeechSynthesisUtterance(data.answer);
    utterance.lang = 'fr-FR';
    utterance.rate = 1.0;
    speechSynthesis.speak(utterance);

    document.getElementById('status').textContent = 'Pret';
};
```

8. Gestion de plusieurs agents

8.1 Configuration des agents

```
# python_backend/agents_config.py

AGENTS_PROMPTS = {
    'agent_commercial': '''
        Tu es un agent commercial vocal.
        Specialise dans les produits et ventes.
        Reponds de facon professionnelle et concise.
        Utilise recherche_sql pour les chiffres de vente.
        Utilise recherche_documents pour les fiches produits.
    ''',
    'agent_support': '''
        Tu es un agent support technique vocal.
        Aide les utilisateurs avec leurs problemes.
        Utilise recherche_documents pour les procedures.
        Utilise recherche_sql pour l'historique des tickets.
    ''',
    'agent_rh': '''
        Tu es un agent ressources humaines vocal.
        Reponds aux questions sur les politiques RH.
        Utilise recherche_documents pour les politiques.
        Utilise recherche_sql pour les donnees employes.
    '''
}
```

9. Tests et validation

9.1 Tests de l'orchestration

Test 1 : Source documents uniquement

```
curl -X POST http://localhost:8000/ask \
  -H 'Content-Type: application/json' \
  -d '{"question": "Que dit le rapport sur la strategie ?",
       "agent_id": "agent_commercial"}'
# Attendu : agent utilise recherche_documents
```

Test 2 : Source SQL uniquement

```
curl -X POST http://localhost:8000/ask \
  -H 'Content-Type: application/json' \
  -d '{"question": "Combien de clients dans la base ?",
       "agent_id": "agent_commercial"}'
# Attendu : agent utilise recherche_sql
```

Test 3 : Source combinee

```
curl -X POST http://localhost:8000/ask \
  -H 'Content-Type: application/json' \
  -d '{"question": "Le produit du rapport, quel est son CA ?",
       "agent_id": "agent_commercial"}'
# Attendu : agent utilise recherche_combinee
```

9.2 Checklist complete

- RAGFlow accessible sur http://localhost
- Documents PDF et Word uploades et vectorises
- Excel converti en tables MySQL dans PHPMyAdmin
- Les 3 tools retournent des resultats
- Router choisit la bonne source selon la question
- recherche_combinee fusionne bien les deux sources
- FastAPI repond sur http://localhost:8000
- Node.js communique correctement avec FastAPI
- STT capture la voix en francais
- TTS prononce la reponse naturellement
- Memoire conservee entre les questions d'une session
- Plusieurs agents ont des comportements distincts

10. Resume — Ordre de realisation

Etape	Action	Duree	Priorite
1	Installer Python + packages	30 min	Critique
2	Installer Docker + lancer RAGFlow	1h	Critique
3	Uploader et vectoriser PDF et Word	30 min	Critique
4	Convertir Excel en MySQL	30 min	Critique
5	Creer tools.py avec 3 tools + orchestration	2h	Critique
6	Creer le System Prompt d'orchestration	1h	Critique
7	Creer main.py FastAPI	1h	Critique
8	Integrer agent.js dans Node.js	30 min	Critique
9	Tester les 3 types de questions	1h	Critique
10	Ajouter multi-agents et memoire	1h	Important
11	Tests finaux et documentation	2h	Recommande

Bonne chance pour ta soutenance !

Node.js + FastAPI + LangChain + RAGFlow + MySQL + Orchestration